

KV Cache Engine

Software Overview

LONGHORN SILICON — COMPILER CO-DESIGN ORIENTATION
Block 2 of 4 — KV Cache Compression / Decompression

Version: 0.1
Date: 2026-05-14
Audience: Compiler team, runtime engineers

Contents

1	Purpose	3
2	What the Block Does	3
2.1	Compression Pipeline	3
3	Reference Models	3
3.1	Directory Layout	3
3.2	Which Models Are Available	4
3.3	Build Everything	4
4	What the Compiler Should Emit	4
4.1	Worked Example: Attention with KV Cache	4
5	Integration Phases	5
6	Interaction with Block 1 (Precision Controller)	5
7	Open Questions for the Compiler Team	6

1. Purpose

This document orients the compiler team on the KV cache engine block. It covers: what the block does, how to use the reference models, what the compiler should emit, and what integration phases look like.

For the formal interface contract, see the ISA specification ([docs/isa/kv_cache_engine_isa.pdf](#)). For the API reference, see [docs/reference_model_api.pdf](#).

2. What the Block Does

The KV cache engine sits between the attention compute unit (block 1) and the memory hierarchy controller (block 4). It compresses key and value vectors before writing to on-chip SRAM, and decompresses them when the attention unit needs them for dot-product computation.

- **On write:** incoming FP16 K/V vectors are compressed via TurboQuant+ (PolarQuant + QJL for keys, PolarQuant only for values) and stored in SRAM.
- **On read:** compressed entries are decompressed back to FP16 coordinates and streamed to the attention unit via AXI-Stream.
- **Eviction:** when SRAM is full, the block signals the memory hierarchy controller to spill entries to DRAM.

2.1 Compression Pipeline

1. **Norm extraction** — L2 norm stored as metadata (16-bit fixed-point)
2. **Rotation** — Walsh-Hadamard Transform + random sign flips (deterministic from seed, self-inverse, $O(n \log n)$)
3. **Quantization** — 3-bit optimal Lloyd-Max scalar quantization (8 centroids symmetric around zero)
4. **QJL projection** (K only) — 1-bit sign projection of the quantization residual (preserves inner products)
5. **Packing** — bit-pack norm + indices + residual norm + signs into a compact SRAM word

Decompression reverses the pipeline: unpack $\rightarrow$ centroid lookup $\rightarrow$ QJL correction (K only) $\rightarrow$ inverse WHT $\rightarrow$ norm rescaling.

3. Reference Models

3.1 Directory Layout

```
sw/
+-- README.md                <- orientation (this doc's markdown
    form)
+-- reference_model/
    +-- README.md            <- API reference + build instructions
    +-- Makefile
```

```

+-- kv_cache_engine_ref.py      <- Python golden reference
+-- kv_cache_engine_ref.hpp     <- C++ header
+-- kv_cache_engine_ref.cpp     <- C++ implementation
+-- test_kv_cache_engine_ref.py
+-- test_kv_cache_engine_ref.cpp

```

3.2 Which Models Are Available

Block	Status	C++ class	C API	Python
KV Cache Engine	Bit-exact vs RTL	lhsi::KVCacheEngine	lhsi_kv_*	KVCacheEngine

3.3 Build Everything

```

cd sw/reference_model
make test-all      # builds C++ tests, runs them, runs Python tests
make shared        # libkv_cache_engine_ref.so
make static        # libkv_cache_engine_ref.a
make clean

```

Requires Python 3.10+, NumPy, and a C++17 compiler.

4. What the Compiler Should Emit

A compiler backend targeting the KV cache engine emits sequences of the operations defined in the ISA spec (§4 of docs/isa/kv_cache_engine_isa.pdf):

Operation	C API	What it does
KV_QUERY	lhsi_kv.info()	Read hardware config
KV_RESET	lhsi_kv.reset()	Clear SRAM, reset FSM
KV_COMPRESS_KEY	lhsi_kv.compress_key()	Compress + store key
KV_COMPRESS_VALUE	lhsi_kv.compress_value()	Compress + store value
KV_DECOMPRESS_KEY	lhsi_kv.decompress_key()	Read + decompress key
KV_DECOMPRESS_VALUE	lhsi_kv.decompress_value()	Read + decompress value

4.1 Worked Example: Attention with KV Cache

```

from kv_cache_engine_ref import KVCacheEngine

engine = KVCacheEngine()

# Store phase: compress new tokens' K/V
for token_idx, (k, v) in enumerate(new_kv_pairs):
    addr = token_idx % engine.info.sram_depth
    engine.store_kv(addr, k, v)

# Attention phase: decompress for Q*K^T
for cached_idx in range(num_cached):
    k_hat = engine.load_k(cached_idx)

```

```

v_hat = engine.load_v(cached_idx)
score = dot(query, k_hat) / sqrt(D)
# Route through precision controller (block 1)
# ...

```

5. Integration Phases

Phase	Timeline	Compiler targets	We provide
0	now	Python + C++ reference models	Models + ISA + tests
1	when FPGA arrives	AXI-Lite + AXI-Stream on Zynq	Vivado bitstream + driver
2	all 4 blocks done	Multi-block project	FPGA Integrated attention pipeline
3	post-tape-out	PCIe-attached chip	TSMC 16FFC silicon + driver

The ISA contract ([docs/isa/kv_cache_engine_isa.pdf](#)) is stable across all four phases. Only the runtime implementation changes — Python function call, AXI register write, or PCIe MMIO.

6. Interaction with Block 1 (Precision Controller)

The KV cache engine and precision controller work together during attention inference:

1. KV cache engine decompresses K/V vectors from SRAM
2. The attention unit computes $Q \cdot K^T$ scores
3. The precision controller decides INT8 vs FP16 per tile
4. The MAC array computes $\text{softmax}(S) \cdot V$ at the chosen precision

A compiler backend that targets both blocks emits interleaved KV_DECOMPRESS_* and PC_PUSH_TILE operations. The reference models for both blocks can be composed in the same process:

```

from kv_cache_engine_ref import KVCacheEngine
from precision_controller_ref import PrecisionController # from block 1

kv = KVCacheEngine()
pc = PrecisionController()

k_hat = kv.load_k(addr)
scores = query @ k_hat # Q * K^T
decision = pc.process_tile(scores)
# decision: True = FP16, False = INT8

```

7. Open Questions for the Compiler Team

1. **Granularity:** does the compiler emit individual compress/decompress ops, or a fused `kv_attention(Q, K_cache, V_cache)` op?
2. **Batch interface:** should the reference model support batched compress/decompress (multiple vectors in one call) for throughput?
3. **Async model:** should `KV_COMPRESS_KEY` be non-blocking with a separate completion query, matching the hardware pipeline?
4. **Cost model:** are placeholder cycle counts sufficient, or do you need post-synthesis numbers for the scheduler?

LonghornSilicon KV Cache Engine — Software Overview. This document is open and may be redistributed.